
APACHE AIRFLOW 3.1.7



STAGE 5 / ADVANCED

Advanced

Event-driven, Dynamic Mapping, Task SDK 3.0 & RAG

Target:	Backend / Distributed Systems Engineer
Duration:	Ngày 14-16 (2-3 giờ/ngày)
Focus:	Cutting-edge features
Version:	Apache Airflow 3.1.7

Mục lục

1. Asset Triggers (Event-driven Scheduling)

2. Dynamic Task Mapping

2.1. expand() và map()

2.2. Kết hợp với API call

3. Airflow 3.0 Task SDK & Edge Executor

3.1. Task SDK cho remote execution

3.2. Edge Executor config

4. DAG Versioning (AIP-66)

5. RAG / AI Pipeline Orchestration

6. Bài tập thực hành

1. Asset Triggers (Event-driven Scheduling)

Airflow 3.x giới thiệu **Asset-driven scheduling** (AIP-74, AIP-75) — cho phép DAG tự động chạy khi 1 asset (dữ liệu) được cập nhật. Đây là bước chuyển từ time-based sang event-based orchestration. Khái niệm "Dataset" trong Airflow 2.x đã được nâng cấp thành "Asset" trong 3.x với decorator `@asset` mới.

```
from airflow.sdk import Asset
from airflow.decorators import dag, task
from datetime import datetime

# Định nghĩa assets
invoice_raw = Asset("redis://invoices/raw/{{ ds }}")
invoice_processed = Asset("redis://invoices/processed/{{ ds }}")

@dag(
    schedule=[invoice_raw], # Trigger khi asset raw được update
    start_date=datetime(2026, 6, 1),
    catchup=False,
    tags=['stage5', 'event-driven'],
)
def process_on_asset_update():

    @task(outlets=[invoice_processed])
    def transform():
        print("Processing because raw asset updated")
        return "done"

    transform()

process_on_asset_update()
```

Bảng 1: So sánh scheduling models trong Airflow 3.x

Model		Trigger	Use case
Time-based	(<code>schedule='@daily'</code>)	Cron — chạy đúng giờ	ETL định kỳ, báo cáo hàng ngày
Asset-based	(<code>schedule=[asset]</code>)	Event — chạy khi data sẵn sàng	Pipeline phụ thuộc data của pipeline khác
External	trigger (<code>TriggerDagRunOperator</code>)	DAG khác yêu cầu	Cross-DAG dependencies

Ưu điểm Asset Triggers

Thay vì chạy DAG theo giờ (có thể chạy khi data chưa sẵn sàng), Asset triggers đảm bảo DAG **chỉ chạy khi dữ liệu thực sự có**. Điều này tiết kiệm tài nguyên và giảm lỗi do thiếu dữ liệu.

2. Dynamic Task Mapping

Dynamic Task Mapping là tính năng mạnh mẽ của Airflow 3.x, cho phép tạo số lượng task động dựa trên dữ liệu runtime. Thay vì định nghĩa 10 task cố định, bạn có thể tạo N task tùy thuộc số lượng file cần xử lý.

2.1. expand() và map()

```

from airflow.decorators import dag, task
from datetime import datetime

@dag(
    schedule='@daily',
    start_date=datetime(2026, 6, 1),
    catchup=False,
    tags=['stage5', 'dynamic'],
)
def dynamic_invoice_processing():

    @task
    def get_files_to_process(ds: str) → list:
        """Lấy danh sách file cần xử lý – trong production: list Redis hoặc query
        DB."""
        return [
            f"redis://{invoices}/{ds}/file_{i}.xml"
            for i in range(1, 6)
        ]

    @task
    def process_file(file_path: str) → dict:
        """Mỗi file = 1 task instance chạy song song."""
        print(f"Processing {file_path}")
        return {"file": file_path, "status": "ok"}

    @task
    def aggregate(results: list) → dict:
        """Gộp kết quả từ tất cả các task process_file."""
        return {"total": len(results), "results": results}

    # Dynamic mapping: mỗi phần tử trong list = 1 task
    files = get_files_to_process("{ ds }")
    processed = process_file.expand(file_path=files)
    aggregate(processed)

dynamic_invoice_processing()

```

2.2. Kết hợp với API call

Dynamic mapping đặc biệt hữu ích khi kết hợp với API call để lấy danh sách items cần xử lý:

```

@task
def fetch_partitions(table: str) → list:
    """Query DB hoặc API để lấy danh sách partitions cần xử lý."""
    return ['2026-06-01', '2026-06-02', '2026-06-03']

@task
def process_partition(partition: str) → dict:
    return {"partition": partition, "rows_processed": 10000}

@task
def report(results: list) → None:
    total = sum(r["rows_processed"] for r in results)
    print(f"Total rows: {total}")

partitions = fetch_partitions("invoices")
results = process_partition.expand(partition=partitions)
report(results)

```

Bảng 2: Các phương thức dynamic mapping

Phương thức	Mục đích	Ví dụ
<code>.expand()</code>	Tạo nhiều task từ 1 list — song song	<code>task.expand(x=[1,2,3])</code>
<code>.map()</code>	Biến đổi output trước khi expand	<code>task.map(lambda x: x*2)</code>
<code>.partial()</code>	Truyền tham số cố định trước expand	<code>task.partial(a=1).expand(b=[2,3])</code>

3. Airflow 3.0 Task SDK & Edge Executor

Airflow 3.0 giới thiệu **Task SDK** (AIP-72) — cho phép viết task mà không cần cài đặt toàn bộ Airflow. Task SDK nhẹ hơn, phù hợp cho chạy task ở remote/edge nodes. Đây là thay đổi kiến trúc lớn nhất: chuyển từ monolith sang service-oriented architecture.

3.1. Task SDK cho remote execution

```
# Task SDK cho remote execution – không cần full Airflow install
# Cài đặt: pip install apache-airflow-task-sdk

from airflow.sdk import task, dag
from datetime import datetime, timedelta

@task
def remote_validation(file_path: str) → bool:
    """Task này chạy ở remote/edge node, chỉ cần airflow-sdk pip package."""
    import xml.etree.ElementTree as ET
    tree = ET.parse(file_path)
    return tree.getroot().tag == 'HDon'

@dag(
    schedule=timedelta(hours=1),
    start_date=datetime(2026, 6, 1),
    catchup=False,
)
def edge_execution_demo():
    remote_validation("/data/invoice.xml")

edge_execution_demo()
```

Import mới trong Airflow 3.x

Airflow 3.x khuyến nghị import từ `airflow.sdk` thay vì các module nội bộ:

```
# Cũ (Airflow 2.x)
from airflow.models import DAG
from airflow.decorators import task

# Mới (Airflow 3.x)
from airflow.sdk import DAG, task, Asset
```

Các import cũ vẫn hoạt động (backward compatibility) nhưng sẽ bị xóa trong tương lai.

3.2. Edge Executor config

Edge Executor (AIP-69) cho phép chạy task trên các node remote/edge, kết nối qua HTTP(s) với API server trung tâm:

```
# airflow.cfg – cấu hình Edge Executor
[core]
executor = airflow.providers.edge3.executors.EdgeExecutor

[edge]
api_enabled = True
api_url = https://edge.airflow.internal
```

Bảng 3: So sánh Task SDK và traditional operators

Tiêu chí	Task SDK	Traditional
Dependency	Chỉ cần <code>apache-airflow-task-sdk</code>	Cần full Airflow
Kích thước	Nhẹ (~MB)	Nặng (~hundreds MB)
Chạy ở đâu	Edge, IoT, remote	Worker cluster
Tính năng	Cơ bản	Đầy đủ operators, hooks
Phù hợp	Task đơn giản, phân tán	Complex pipeline

Edge Worker Deployment

Edge Worker là process nhẹ chạy trên edge device, pull task từ API server qua HTTP(s). Cài đặt:

```
pip install apache-airflow-providers-edge3
airflow edge worker --api-url https://edge.airflow.internal
```

Edge Worker gửi heartbeat định kỳ về API server. Hiện tại scale được ~80 workers. Mỗi worker pull task và gửi kết quả về qua REST API.

4. DAG Versioning (AIP-66)

Airflow 3.x hỗ trợ **DAG Versioning** — theo dõi thay đổi cấu trúc DAG theo thời gian. Điều này cho phép:

- Xem định nghĩa DAG tại thời điểm chạy (historical DAG definition)
- So sánh các phiên bản DAG qua UI
- Debug dễ dàng hơn khi DAG thay đổi giữa các run


```
# Kiểm tra version DAG qua CLI
airflow dags reserialize

# Xem version trong UI: Browse → DAG Versions
```

5. RAG / AI Pipeline Orchestration

Airflow có thể orchestrate các pipeline AI/ML, bao gồm RAG (Retrieval-Augmented Generation) — một kiến trúc phổ biến cho ứng dụng AI.

```
from airflow.decorators import dag, task
from datetime import datetime

@dag(
    schedule='@daily',
    start_date=datetime(2026, 6, 1),
    catchup=False,
    tags=['stage5', 'ai', 'rag'],
)
def rag_document_pipeline():

    @task
    def extract_docs() → list:
        """Trích xuất danh sách tài liệu cần xử lý."""
        return ["doc1.pdf", "doc2.pdf", "doc3.pdf"]

    @task
    def chunk_and_embed(doc: str) → dict:
        """Chia nhỏ tài liệu và tạo vector embeddings.
        Trong production: gọi local LLM (llama.cpp/Qwen) hoặc API."""
        return {"doc": doc, "chunks": 10, "vectors": "..."}

    @task
    def index_to_vector_db(data: dict) → None:
        """Lưu vector embeddings vào vector database."""
        print(f"Indexing {data['doc']} with {data['chunks']} chunks")

    # Luồng: extract → parallel chunk/embed → parallel index
    docs = extract_docs()
    embedded = chunk_and_embed.expand(doc=docs)
    index_to_vector_db.expand(data=embedded)

rag_document_pipeline()
```

Airflow trong AI/ML Pipeline

Airflow không thay thế training framework (PyTorch, TensorFlow), nhưng là **orchestrator xuất sắc** cho: data preparation, model training jobs, evaluation, deployment, và monitoring. Dynamic Task Mapping đặc biệt phù hợp cho xử lý batch của nhiều documents/images. Airflow 3.x còn hỗ trợ `logical_date=None` (AIP-83) cho các workflow không cần lịch như model inference.

6. Bài tập thực hành

Bảng 4: Bài tập Stage 5

STT	Bài tập	Kỹ năng
1	Tạo Asset trigger DAG: DAG A update asset → DAG B tự động chạy	Event-driven scheduling
2	Viết dynamic task mapping: 5 files → 5 parallel tasks → aggregate	Dynamic mapping
3	Thử Airflow 3.0 Task SDK: cài <code>pip install apache-airflow-task-sdk</code> , viết remote task	Task SDK
4	Tạo RAG pipeline: extract PDF → chunk → embed (mock) → index	AI orchestration
5	Test <code>airflow dags reserialize</code> để verify DAG versioning	DAG management
6	Thử <code>airflow db migrate</code> với Alembic revision check	DB migration